

Reinforcement Learning: Policy Iteration, Learning from Data, and Exploration

Stanford Lecture Notes

Abstract

These comprehensive notes cover advanced topics in reinforcement learning, including the policy iteration algorithm, learning state transition probabilities from data, the complete RL workflow, and the exploration-exploitation tradeoff. Topics include policy iteration vs. value iteration, maximum likelihood estimation of transition probabilities, epsilon-greedy exploration, and Boltzmann exploration.

Contents

1	Policy Iteration Algorithm	3
1.1	Algorithm Description	3
1.2	Key Differences from Value Iteration	4
1.3	Policy Evaluation Step (Detail)	4
2	Value Iteration vs. Policy Iteration	4
2.1	Comparative Analysis	4
2.2	Convergence Properties	5
2.2.1	Value Iteration	5
2.2.2	Policy Iteration	5
2.3	Practical Considerations	5
3	Learning State Transition Probabilities	6
3.1	Motivation	6
3.2	Maximum Likelihood Estimation	6
3.2.1	Handling Edge Cases	6
3.2.2	Laplace Smoothing (Optional)	7
3.3	Estimating Reward Functions	7
4	Complete Reinforcement Learning Workflow	7
4.1	Algorithm: Model-Based RL	7
4.2	Why This is Called "Model-Based" RL	7
5	Exploration vs. Exploitation	7
5.1	The Exploration-Exploitation Tradeoff	7
5.2	Motivating Example	8
5.3	Real-World Application: Online Advertising	9

6	Exploration Strategies	9
6.1	Epsilon-Greedy Exploration	9
6.1.1	Mathematical Formulation	10
6.1.2	Properties	10
6.1.3	Naming Note	10
6.2	Decay Schedules for Epsilon	11
6.2.1	Linear Decay	11
6.2.2	Exponential Decay	11
6.3	Boltzmann Exploration	11
6.3.1	Temperature Parameter	11
6.3.2	Advantages over Epsilon-Greedy	11
6.3.3	When to Use Boltzmann	12
6.4	Comparison of Exploration Strategies	12
7	Advanced Exploration Topics	12
7.1	Intrinsic Motivation	12
7.1.1	Common Intrinsic Reward Formulations	12
8	Practical Implementation Considerations	13
8.1	How Often to Update the Policy?	13
8.2	Convergence with Exploration	13
9	Complete Algorithm Summary	13
10	Summary and Key Takeaways	13
10.1	Main Concepts	13
10.2	Practical Guidelines	15
10.3	Next Topics	15
11	Additional Resources	15

1 Policy Iteration Algorithm

Policy iteration is an alternative algorithm to value iteration for solving MDPs. While value iteration focuses on computing the optimal value function V^* , policy iteration focuses directly on improving the policy.

1.1 Algorithm Description

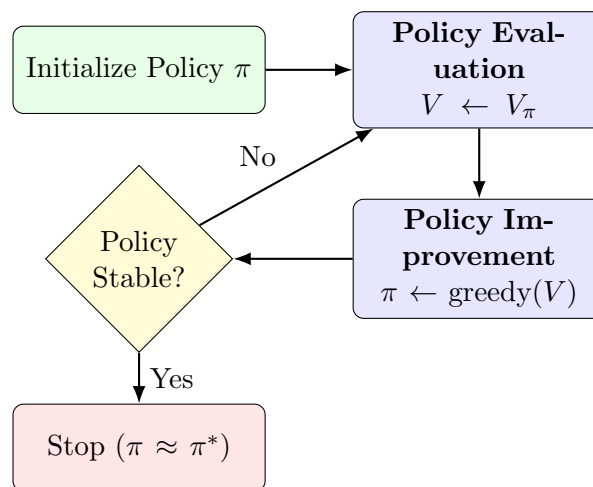


Figure 1: The Policy Iteration Loop. The algorithm alternates between evaluating the current policy (calculating V_π) and improving it (making it greedy with respect to V_π).

Policy Iteration Algorithm

Initialize: Choose random policy π (for each state s , select random action)

Repeat until convergence:

1. **Policy Evaluation:** Solve for V_π using Bellman's equation:

$$V_\pi(s) = R(s) + \gamma \sum_{s'} P_{s,\pi(s)}(s') \cdot V_\pi(s')$$

This is a linear system of $|S|$ equations with $|S|$ unknowns.

2. **Policy Improvement:** Update policy using:

$$\pi(s) := \arg \max_a \sum_{s'} P_{sa}(s') \cdot V_\pi(s')$$

Output: Optimal policy π^* and value function V^*

1.2 Key Differences from Value Iteration

Value Iteration	Policy Iteration
Focuses on value function V	Focuses on policy π
Updates V iteratively using max operator	Solves exact V_π for current policy
Computes policy only at the end	Updates policy on every iteration
No linear system to solve	Solves linear system in each iteration
Converges to V^* asymptotically	Reaches exact V^* in finite iterations

1.3 Policy Evaluation Step (Detail)

Given a fixed policy π , we need to solve for $V_\pi(s)$ for all states s .

Linear System Formulation:

For each state s , we have:

$$V_\pi(s) = R(s) + \gamma \sum_{s'} P_{s,\pi(s)}(s') \cdot V_\pi(s')$$

This can be written in matrix form:

$$\mathbf{V}_\pi = \mathbf{R} + \gamma \mathbf{P}_\pi \mathbf{V}_\pi$$

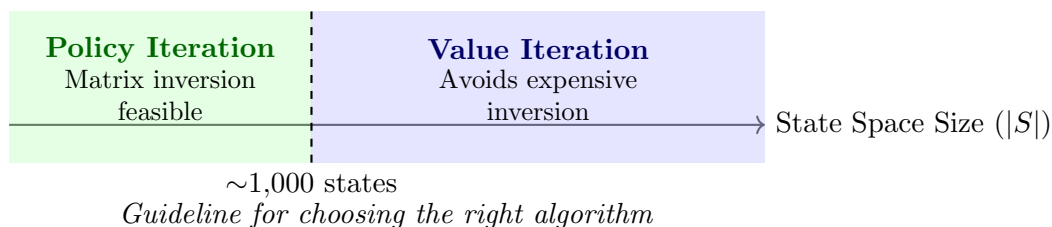
Solving for $\mathbf{V}_\pi$:

$$\mathbf{V}_\pi = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1} \mathbf{R}$$

Computational Cost: Matrix inversion costs $O(|S|^3)$ time.

2 Value Iteration vs. Policy Iteration

2.1 Comparative Analysis



When to Use Each Algorithm

Use Policy Iteration when:

- State space is relatively small ($|S| \lesssim \text{few hundred}$)
- Matrix inversion is computationally feasible
- You want exact convergence in finite iterations
- Faster convergence (fewer iterations) is important

Use Value Iteration when:

- State space is large ($|S| \gtrsim 10,000$)
- Matrix inversion is expensive or infeasible
- Asymptotic convergence is acceptable
- Simpler implementation is preferred

2.2 Convergence Properties

2.2.1 Value Iteration

- Converges **asymptotically** to V^*
- Never reaches exact V^* , but gets arbitrarily close
- Geometric (exponential) convergence rate
- Similar to gradient descent in supervised learning
- Error decreases by factor γ per iteration

2.2.2 Policy Iteration

- Converges in **finite iterations**
- Reaches exact optimal policy π^* and value V^*
- Policy stops changing after finite iterations
- Similar to normal equations in linear regression
- Generally requires fewer iterations than value iteration

2.3 Practical Considerations

Computational Complexity:

Value Iteration:

- Per iteration: $O(|S|^2|A|)$ for dense transition matrices
- No matrix inversion required

- More iterations needed

Policy Iteration:

- Per iteration: $O(|S|^3)$ for matrix inversion
- Fewer iterations needed
- Total cost depends on sparsity of problem

Empirical Observation: Value iteration is more commonly used in practice, especially for larger problems.

3 Learning State Transition Probabilities

In many real-world applications, the state transition probabilities $P_{sa}(s')$ are not known in advance and must be learned from data.

3.1 Motivation

- Real robots have uncertain dynamics
- Helicopter control surfaces respond unpredictably
- Chess opponents have unknown strategies
- Environment dynamics may be too complex to model analytically

Example: We assumed a robot going North has 0.8 probability of success, but in reality these probabilities must be estimated from experience.

3.2 Maximum Likelihood Estimation

Estimating Transition Probabilities

$$\hat{P}_{sa}(s') = \frac{\text{Count}(s, a, s')}{\text{Count}(s, a)}$$

where:

- $\text{Count}(s, a, s')$ = number of times action a in state s led to state s'
- $\text{Count}(s, a)$ = number of times action a was taken in state s

3.2.1 Handling Edge Cases

Zero Division Problem: What if we've never tried action a in state s ?

Common Heuristic: If $\text{Count}(s, a) = 0$, set:

$$\hat{P}_{sa}(s') = \frac{1}{|S|}$$

This assumes uniform distribution over all states (maximum uncertainty).

3.2.2 Laplace Smoothing (Optional)

Laplace smoothing can also be applied:

$$\hat{P}_{sa}(s') = \frac{\text{Count}(s, a, s') + 1}{\text{Count}(s, a) + |S|}$$

Note: Unlike Naive Bayes, MDP solvers are not very sensitive to zero probabilities, so Laplace smoothing is **not commonly used** in practice for reinforcement learning.

3.3 Estimating Reward Functions

While reward functions are typically specified as part of the problem, sometimes they must also be learned:

Scenarios requiring reward estimation:

- Stock trading: returns are random and state-dependent
- Self-driving cars: penalties for hitting potholes
- Variable environment conditions

Estimation: Compute empirical average reward:

$$\hat{R}(s) = \frac{1}{n} \sum_{i=1}^n r_i$$

where r_i are observed rewards when visiting state s .

4 Complete Reinforcement Learning Workflow

4.1 Algorithm: Model-Based RL

4.2 Why This is Called "Model-Based" RL

This approach is called **model-based** because:

- We explicitly learn a **model** of the environment: $\hat{P}_{sa}$
- We use this model to plan (solve the MDP)
- Contrast with **model-free** methods (Q-learning, policy gradients)

5 Exploration vs. Exploitation

5.1 The Exploration-Exploitation Tradeoff

Critical Problem

Exploitation: Take actions that maximize expected reward based on current knowledge

Exploration: Take actions to gather new information, even if they seem suboptimal

Tradeoff: How to balance between maximizing immediate rewards and discovering potentially better strategies?

Algorithm 1 Model-Based Reinforcement Learning

- 1: **Initialize:** Set initial policy π (e.g., random)
- 2: **repeat**
- 3: **Step 1: Gather Experience**
- 4: Execute policy π in the MDP to collect state-action-nextstate tuples (s, a, s')
- 5: **Step 2: Update Model**
- 6: Update estimates of state transition probabilities:

$$\hat{P}_{sa}(s') = \frac{\text{Count}(s, a, s')}{\text{Count}(s, a)}$$

- 7: **Step 3: Solve MDP**
- 8: Use value iteration (or policy iteration) to compute optimal policy given current estimates:

$$V^*(s) = R(s) + \max_a \gamma \sum_{s'} \hat{P}_{sa}(s') \cdot V^*(s')$$

- 9: **Step 4: Update Policy**

$$\pi(s) := \arg \max_a \sum_{s'} \hat{P}_{sa}(s') \cdot V^*(s')$$

- 10: **until** convergence

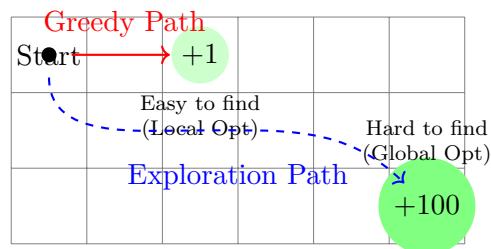


Figure 2: The Trap of Pure Exploitation. A greedy agent will consistently choose the small nearby reward (+1) and never discover the massive reward (+100) further away.

5.2 Motivating Example

Scenario:

- Robot starts near top-left
- Small penalty of -0.02 per step (fuel cost)
- +1 reward is closer and easier to find
- +100 reward is farther away

Problem: If the robot randomly finds +1 first, a purely greedy algorithm will:

- Conclude that going to +1 is optimal
- Never explore the rest of the map

- Never discover the +100 reward
- Converge to a **local optimum**

5.3 Real-World Application: Online Advertising

Problem Setting:

- Ad platform has many ads to show users
- Platform doesn't know which ads users will click
- Each impression is an opportunity to learn or earn

Pure Exploitation:

- Always show ads with highest predicted click-through rate
- Maximizes short-term revenue
- Never learns about new ads or changing user preferences
- May miss better long-term opportunities

With Exploration:

- Sometimes show ads with uncertain click probability
- Learn which ads resonate with which users
- Discover unexpected interests (e.g., Mars landers!)
- Improve long-term matching quality
- Better overall system performance

Real Implementation: Major ad platforms (Google, Facebook) explicitly use exploration strategies in their ranking systems.

6 Exploration Strategies

6.1 Epsilon-Greedy Exploration

Epsilon-Greedy Policy

Algorithm: At each time step:

- With probability $(1 - \varepsilon)$: Execute best known policy $\pi(s)$
- With probability ε : Take random action

Typical values: $\varepsilon = 0.1$ (explore 10% of the time)

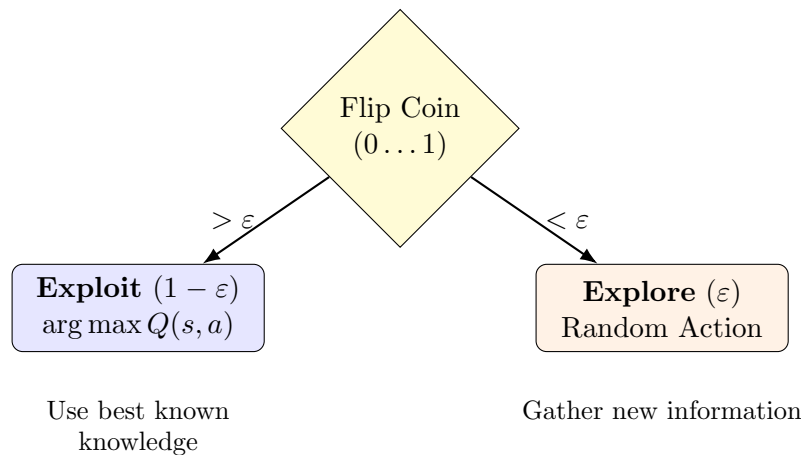


Figure 3: Decision logic for Epsilon-Greedy Exploration. Most of the time we exploit, but occasionally we explore randomly.

6.1.1 Mathematical Formulation

$$a_t = \begin{cases} \pi(s_t) & \text{with probability } 1 - \epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases}$$

6.1.2 Properties

Advantages:

- Simple to implement
- Guarantees all state-action pairs will be visited
- Converges to optimal policy for any finite MDP
- Most commonly used exploration strategy

Disadvantages:

- Explores uniformly (doesn't prioritize promising actions)
- May take long time if optimal path is hard to find randomly
- Fixed ϵ continues exploring even after learning

6.1.3 Naming Note

Terminology Quirk: The name "epsilon-greedy" is somewhat misleading:

- ϵ is the probability of **exploring** (acting randomly)
- $(1 - \epsilon)$ is the probability of being greedy
- More logical name would be " $(1 - \epsilon)$ -greedy" or " ϵ -exploratory"
- But "epsilon-greedy" is the standard term used in literature

6.2 Decay Schedules for Epsilon

Rather than keeping ε constant, common practice is to **decay** it over time:

6.2.1 Linear Decay

$$\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 - \alpha t)$$

6.2.2 Exponential Decay

$$\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \cdot \gamma^t)$$

Intuition:

- Early in learning: explore aggressively (large ε)
- Later in learning: exploit more (small ε)
- Eventually: mostly exploit with minimal exploration

6.3 Boltzmann Exploration

Boltzmann Exploration (Softmax)

Instead of uniform random exploration, choose actions proportionally to their estimated values:

$$P(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

where:

- $Q(s, a) = \sum_{s'} P_{sa}(s') V^*(s')$ (expected value of action a in state s)
- τ is the **temperature** parameter controlling randomness

6.3.1 Temperature Parameter

- **High** τ ($\tau \rightarrow \infty$): Uniform exploration (nearly random)
- **Low** τ ($\tau \rightarrow 0$): Greedy exploitation (always best action)
- **Medium** τ : Probabilistic selection favoring good actions

6.3.2 Advantages over Epsilon-Greedy

- **Informed exploration:** Explores promising actions more than bad ones
- **Smooth policy:** No sharp discrete choice between explore/exploit
- **Value-based:** Exploration proportional to value differences

Example: If $V(s_{\text{north}}) = 10$ and $V(s_{\text{south}}) = 1$:

- Epsilon-greedy: 50% chance of each during exploration
- Boltzmann: Much higher probability of going north

6.3.3 When to Use Boltzmann

- When you have good value estimates
- When action values differ significantly
- When you want more sophisticated exploration
- Often used in deep reinforcement learning

6.4 Comparison of Exploration Strategies

Strategy	Simplicity	Efficiency	Common Use
Epsilon-Greedy	High	Medium	Very Common
Boltzmann	Medium	High	Common
Decaying Epsilon	Medium	High	Common

7 Advanced Exploration Topics

7.1 Intrinsic Motivation

Intrinsic Rewards

Idea: Reward the agent for discovering new states or experiences

Intrinsic Reward Function:

$$R_{\text{total}}(s) = R_{\text{external}}(s) + \beta \cdot R_{\text{intrinsic}}(s)$$

where $R_{\text{intrinsic}}(s)$ measures novelty or information gain.

7.1.1 Common Intrinsic Reward Formulations

1. Visit Count:

$$R_{\text{intrinsic}}(s) = \frac{1}{\sqrt{\text{Count}(s)}}$$

Reward decreases as state is visited more

2. Prediction Error:

$$R_{\text{intrinsic}}(s) = \|\hat{s}' - s'\|^2$$

Reward for states where our model is inaccurate

3. Information Gain: Reward based on reduction in uncertainty about environment

Research Area: Active area of research, pioneered by work on "intrinsic motivation" by researchers at MIT and later expanded by DeepMind and others.

Key Papers: Search for "intrinsic motivation reinforcement learning" for foundational work.

8 Practical Implementation Considerations

8.1 How Often to Update the Policy?

Question: How many actions should we execute before re-solving the MDP?

Answer: There's no harm in updating as frequently as possible!

Practical Considerations:

- **Real robots:** Limited by data collection speed
 - Example: Fly helicopter during day, process data at night
 - Update policy when enough new data is collected
- **Simulation:** Can update continuously
 - No barrier to frequent updates
 - More frequent updates $\rightarrow$ better performance
 - Only constraint is computational cost

8.2 Convergence with Exploration

Theoretical Guarantee

Model-based RL with epsilon-greedy exploration converges to the optimal policy π^* for any finite discrete-state MDP.

Caveat: Convergence may be slow if:

- Optimal path is hard to find randomly
- State space is very large
- ε is too small

9 Complete Algorithm Summary

10 Summary and Key Takeaways

10.1 Main Concepts

1. Policy Iteration:

- Alternates between policy evaluation and policy improvement
- Solves linear systems for exact V_π
- Converges in finite iterations
- Best for small state spaces

2. Value Iteration vs. Policy Iteration:

- Value iteration: asymptotic convergence, no matrix inversion
- Policy iteration: finite convergence, requires matrix inversion

Algorithm 2 Model-Based RL with Exploration

```

1: Initialize: Random policy  $\pi$ , empty counts  $\text{Count}(s, a, s') = 0$ 
2: Set: Exploration parameter  $\varepsilon$  (e.g., 0.1)
3: repeat
4:   Execute epsilon-greedy policy:
5:   Observe current state  $s$ 
6:   With probability  $1 - \varepsilon$ :  $a \leftarrow \pi(s)$ 
7:   With probability  $\varepsilon$ :  $a \leftarrow$  random action
8:   Execute action  $a$ , observe next state  $s'$  and reward  $r$ 
9:   Update counts:
10:   $\text{Count}(s, a, s') \leftarrow \text{Count}(s, a, s') + 1$ 
11:   $\text{Count}(s, a) \leftarrow \text{Count}(s, a) + 1$ 
12:  Update model:
13:   $\hat{P}_{sa}(s') \leftarrow \frac{\text{Count}(s, a, s')}{\text{Count}(s, a)}$  for all  $s, a, s'$ 
14:  Solve MDP: Run value iteration using  $\hat{P}_{sa}$  to get  $V^*$ 
15:  Update policy:
16:   $\pi(s) \leftarrow \arg \max_a \sum_{s'} \hat{P}_{sa}(s') V^*(s')$  for all  $s$ 
17: until convergence or maximum iterations

```

- Choice depends on state space size

3. Learning from Data:

- Estimate P_{sa} using maximum likelihood
- Use counts from observed transitions
- Handle edge cases with uniform priors

4. Exploration-Exploitation:

- Critical for finding globally optimal policies
- Pure exploitation can get stuck in local optima
- Epsilon-greedy is simple and effective
- Boltzmann exploration is more sophisticated

5. Complete RL Workflow:

- Gather experience by executing current policy
- Update model estimates from data
- Solve MDP using value/policy iteration
- Update policy based on new value function
- Repeat until convergence

10.2 Practical Guidelines

Implementation Recommendations

For Small Problems ($|S| < 1000$):

- Use policy iteration for faster convergence
- Epsilon-greedy with $\varepsilon = 0.1$ is sufficient

For Large Problems ($|S| > 10,000$):

- Use value iteration (no matrix inversion)
- Consider Boltzmann exploration
- Use decaying ε schedule

For Real Robots:

- Update policy after collecting batches of data
- Use higher exploration initially
- Monitor safety constraints carefully

10.3 Next Topics

Future lectures will cover:

- Continuous state MDPs
- Function approximation for value functions
- Model-free reinforcement learning (Q-learning)
- Deep reinforcement learning
- Policy gradient methods

11 Additional Resources

Recommended Reading:

- Sutton & Barto: "Reinforcement Learning: An Introduction" (Chapter 4: Dynamic Programming)
- Russell & Norvig: "Artificial Intelligence: A Modern Approach" (Chapter on MDPs)
- Search "intrinsic motivation reinforcement learning" for exploration research